

Quadratic Assignment Problem - Local Search and Random Methods

Aliaksandr Kladneu, Master's student, PUT

April 17, 2026

1 Introduction

The quadratic assignment problem (QAP) is one of the fundamental combinatorial optimization problems in optimization and operations research.

It can be described as follows: given a set of n facilities and n locations, minimize the sum of distances multiplied by flow between facilities (cost function), by assigning facilities to a dedicated location. Mathematically this can be represented as [1]:

$$\min_{\pi} \leftarrow c(\pi) := \sum a_{ij} b_{\pi(i)\pi(j)}$$

where $A = (a_{ij}) \geq 0$ - matrix of distances between locations and $B_{\pi} = (b_{\pi(i)\pi(j)})$ - is a permutation π of rows and columns of matrix of flows between facilities $B = (b_{ij}) \geq 0$, and $c(\pi)$ is the cost of permutation π , i.e. the value of the objective function for the corresponding assignment.

This is an **NP-hard** problem since the solution space has $n!$ permutations, and even small changes to an assignment affect many terms in the objective value, thus yielding a lot of local optima. Thus it's usually not feasible to find an optimal solution for large n .

The applications of the problem include circuit design (placing interconnected electronic components onto a printed circuit board) and flow problems (optimization of flows between facilities) [2]. QAP also generalizes TSP and many other standard combinatorial optimization problems.

1.1 QAP instances

The following instances were chosen, which are available in [3]:

With known optimal solution:

- esc16d
- bur26h
- esc32e
- lipa40a
- lipa80b

With best solution found using heuristics (gap between known lower bound and best cost %):

- tai80b (12.28%)
- wil100 (3.15%), tai100b (10.78%)
- tai150b (11.45%), tho150 (6.3%)
- tai256c (2.03%)

The problems of different size were chosen to investigate the impact of problem size n on the quality of solutions and performance of algorithms. Additionally, "twin" problems were added to investigate how a problem's structure influences these characteristics.

Worth noticing that two of these problems appeared to be "flat", that is sparse or mostly constant, which means one of two things: it's very easy to construct almost an ideal solution (minimal cost, *esc32e*) or it's unfeasible or impossible to decrease the cost (*lipa40*).

1.1.1 Description of the problems

esc-x problems correspond to the circuit testing problem. *tai-b* are uniformly generated random problems, *tai-c* is found in analysis of grey patterns. *lipa-x* are generated asymmetric problems with known optimal solutions. *bur-x* problem represents the time and distance of typing on a keyboard. *tho-x* and *wil-x* problems have rectangular distances.

1.2 Implementation

The implementation is in C++ (20) and is modular: data loading, problem representation, permutation operations, cost/delta evaluation, search algorithms, quality metrics, and statistics, plotting are separated into dedicated components.

Several methods are implemented and compared: heuristic local search (**H**), steepest local search (**S**), greedy local search (**G**), random search (**RS**), and random walk (**RW**). For convenience, they will be referred according to the indicated abbreviations.

Heuristic algorithm (**H**) is defined as follows:

1. match the lowest and the largest elements in matrices
2. swap if it improves the cost
3. search for a new pair in submatrices
4. repeat until no improving moves are available/max number of iterations reached.

In practice this algorithm terminates very fast, in milliseconds even for the largest problem instance. It can be further improved by matching not particular elements, but whole rows/columns.

10 runs of each method were executed in parallel on 10 threads of machine with 32 logical processors, using OpenMP library. Each thread executes the problems in the same order, with random generators independent for each thread. Thus corresponding "wall-clock" timing function from the library is used to measure the "practical" time of the execution as opposed to measuring cpu time.

Methods in *statistics* module aggregate data from multiple experiments and provide functions for plotting using a Matplotlib wrapper [4].

2 Neighbourhood

Neighbourhood operator used in **H**, **S**, **G**, **RW** - pairwise $swap(i, j)$ on a permutation. A neighbor is obtained by selecting two positions i and j and exchanging the assigned locations/facilities at those positions.

The neighbourhood size is the number of possible swaps - number of unique unordered pairs, which is calculated as:

$$|N(x)| = C_2^N = \frac{n(n-1)}{2}$$

Neighbor cost calculation in **H**, **S**, **G**, **RW** is done using a delta update: instead of recomputing the full QAP objective for each neighbor, one computes the change in objective after swapping i and j using a delta formula [5]. If π is the current solution and π' is the solution after $swap(i, j)$, then

$$c(\pi') = c(\pi) + \Delta_{ij}$$

where Δ_{ij} is computed from terms involving rows/columns i, j and $\forall k \neq i, j$. This reduces one neighbor evaluation from full objective recomputation to an incremental update, making local-search iterations significantly faster (especially for larger n).

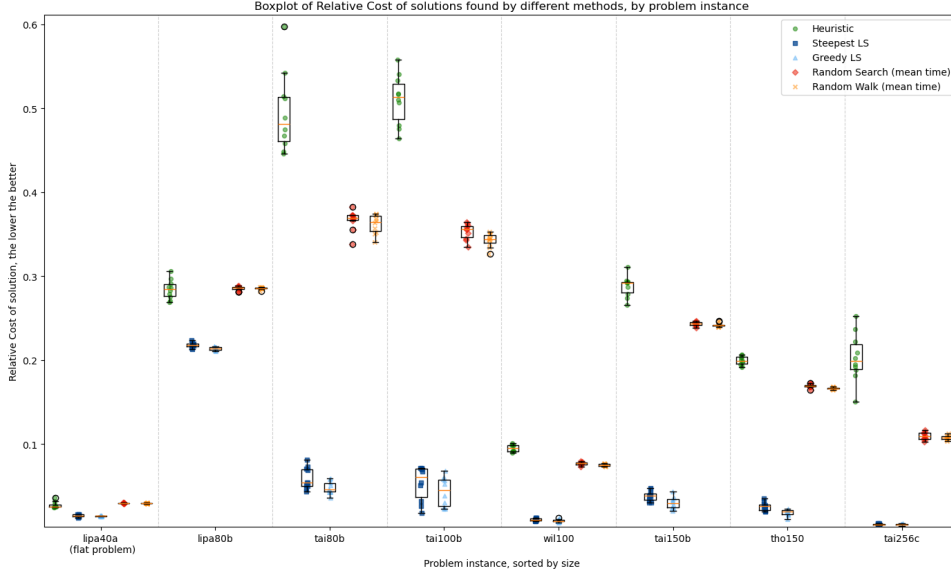


Figure 1: (truncated) Relative cost

3 Comparison of the performance of 5 algorithms on all problem instances

3.0.1 Reporting results and statistical considerations

The comparison of algorithms is mainly done using box-plots for different methods and problems. Since for some statistics (*time*, *norm_cost*) distributions can't have negative values and high deviations in the results are expected, the median value with IQRs are reported as box-plots instead of reporting mean values with confidence intervals.

When providing the plots, the results are sorted by the size of the problem, if not specified otherwise.

3.1 Quality

The first quality metric for comparison of the algorithms is the *relative cost* defined as

$$rel_cost(\pi) := \frac{c(\pi) - c_{best}}{c_{best}} \in [0; +\infty)$$

where π is a solution found by an algorithm, c_{best} is the best cost - cost of the best known solution π_{best} . One can interpret this metric as a (percentage) gap between the cost of found solution and the best known cost: the lower the value the better the quality of the solution regarding to the best known cost.

The objective function $c(\pi)$ can be thought of as Frobenious scalar product of matrices A and B_π , which following Cauchy–Bunyakovsky–Schwarz inequality, should yield the following definition of the secondary quality metric - *normalized cost*:

$$norm_cost(\pi) := \frac{c(\pi)}{\|A\|_F \|B_\pi\|_F} \equiv \frac{\langle A, B_\pi \rangle_F}{\|A\|_F \|B_\pi\|_F} \equiv \frac{\langle A, B_\pi \rangle_F}{\|A\|_F \|B\|_F} \in [0; 1] \quad (\forall A, B \geq 0)$$

This is referred to as *cos quality* in the code since it's equal to the cosine of the angle between matrices in Frobenius terms. This is useful metric since it does not require the "ground truth" optimal value for comparison and allows direct comparison of different problems, keeping values below 1.

Visual examination of the plots shows significant superiority of both **G** and **S** algorithms to solve the problem against proposed heuristic **H** and random algorithms **RW**, **RS**. In all of the given problem

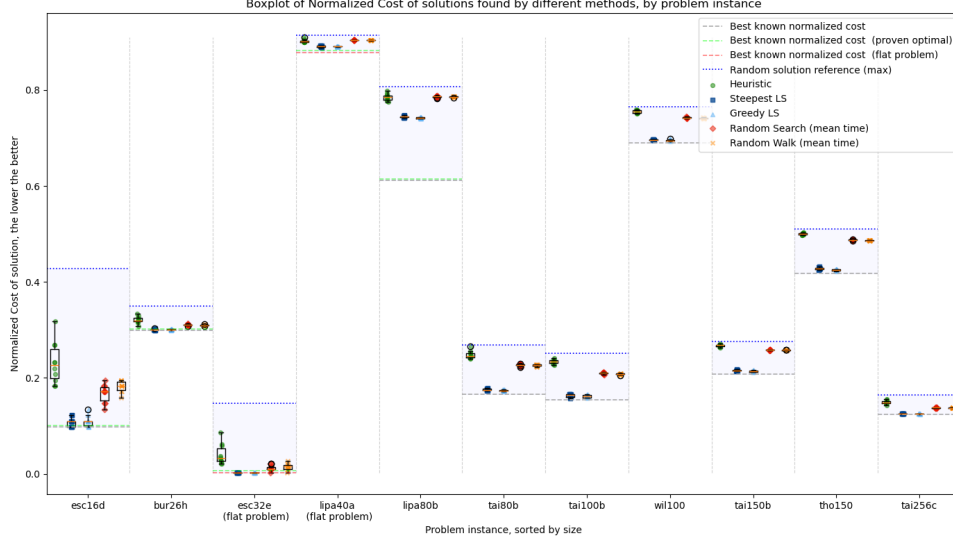


Figure 2: Normalized cost

instances but one (lipa80b) **G** and **S** achieve best known solution with lesser than 10% precision (Figure 1), where **G** demonstrates slightly better quality values probably due to better exploration properties.

3.2 Running time

As expected, time taken by **H** is the lowest one, while **S** requires the most time to thoroughly examine all the solutions in the neighbourhood, and **G** balances quality and time consumption. Random methods were given mean time of the runs of **G** and **S**.

3.3 Efficiency

I introduce two metrics of efficiency which both characterize how fast an algorithm can reduce relative or normalized cost over time. This characteristic - the relative or normalized difference between costs at two consecutive timepoints - over time, should be then calculated as a single number, which is area under the curve divided by time. To visualise the use of this characteristic, one can compare the simple heuristic algorithm which can immediately decrease the cost in much shorter time against a random algorithm, which decrease-time curve would look more like noisy straight line.

$$rel_eff_T := \frac{1}{T} \sum_{i=0}^L [t_{i+1} - t_i] \cdot rel_cost(\pi_i) \equiv \frac{1}{T \cdot c_{best}} \sum_{i=0}^L [t_{i+1} - t_i] \cdot c(\pi_i) - 1$$

The formula for *normalized efficiency* is similar but subtracts normalized best cost

$$\begin{aligned} norm_eff_T &:= \frac{1}{T} \sum_{i=0}^L [t_{i+1} - t_i] \cdot norm_cost(\pi_{t_i}) - norm_cost(\pi_{best}) \equiv \\ &\equiv \frac{1}{T \cdot \|A\|_F \|B\|_F} \sum_{i=0}^L [t_{i+1} - t_i] \cdot c(\pi_i) - \frac{c_{best}}{\|A\|_F \|B\|_F} \end{aligned}$$

where π_i is a solution (permutation) found and accepted at time t_i ; $t_0 = 0$, $c(\pi_0)$ - cost of the initial (random) permutation, $T = \sum_{i=0}^{L+1} [t_i - t_{i-1}] \equiv t_{L+1}$ - time interval between the beginning and the end of a single experiment (run) of an algorithm for a given problem instance, and t_i is a time moment when the cost changes (and this change is accepted), L - number of such changes during the

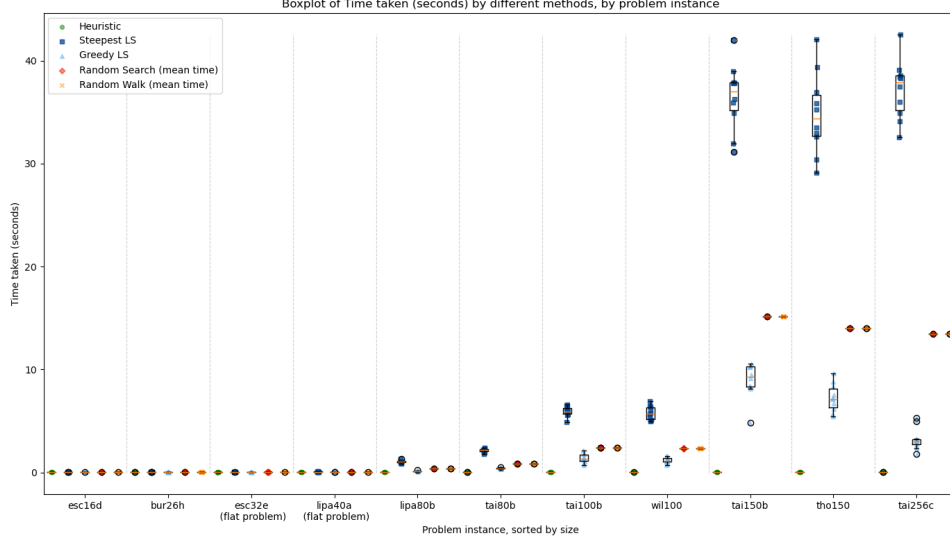


Figure 3: Running time

experiment, t_L - time of the last cost change, $t_{L+1} \equiv T$ - time of the end of an experiment (or time budget). Since both metric indicate how fast the cost decreases, *the lower the value of these metrics the better*.

It should be noted that this metric allows comparison only when time intervals are identical or very close, and thus time budgets should be the same for such comparison. Below are results for different time budgets determined based on running time of **H**, **G** and **S** respectively.

For visual comparison the second metric is more suitable since it keeps plots strictly below 1 on y axis.

Below is the comparison of efficiency of all the 5 algorithms given "strict" (determined by running time of **H**) and "generous" (determined by running time of **S**) time budgets. In the second efficiency analysis **H** is given for reference only and shall not be analysed since proposed efficiency metrics require an algorithm to operate on the whole time interval, while **H** always terminates early and cannot show further improvements, while demonstrating inferior results almost always.

The figures demonstrate superiority of **G** over the rest of the algorithms in both time budget settings, even though "strict" time budget is determined by running time of **H**, and "generous" time budget is determined by running time of **S**, making these conditions sub-optimal for **G**.

3.4 Total number of algorithm steps

The **step** is determined as a swap of two elements in the current solution (permutation). **G** performs much more swaps than **S**, and thus shows that exploratory behaviour (driven by choices that are not necessarily locally optimal) is beneficial when solving all the problems, given the fact that **G** also showed superior quality.

3.5 Total number of evaluated solutions

An evaluated (fully or partially) solution is a solution (permutation) for which a cost value were obtained. Despite the visual discrepancy between total number of evaluated solutions between **S** and **G**, the former evaluates all of the solutions in a current neighbourhood, while **G** evaluates solutions on a much more distant scale given the same processing time, though in more stochastic manner. **RS** seems to evaluate orders of magnitude lower solutions due to the costly evaluations of the objective function (cost) of each random solution.

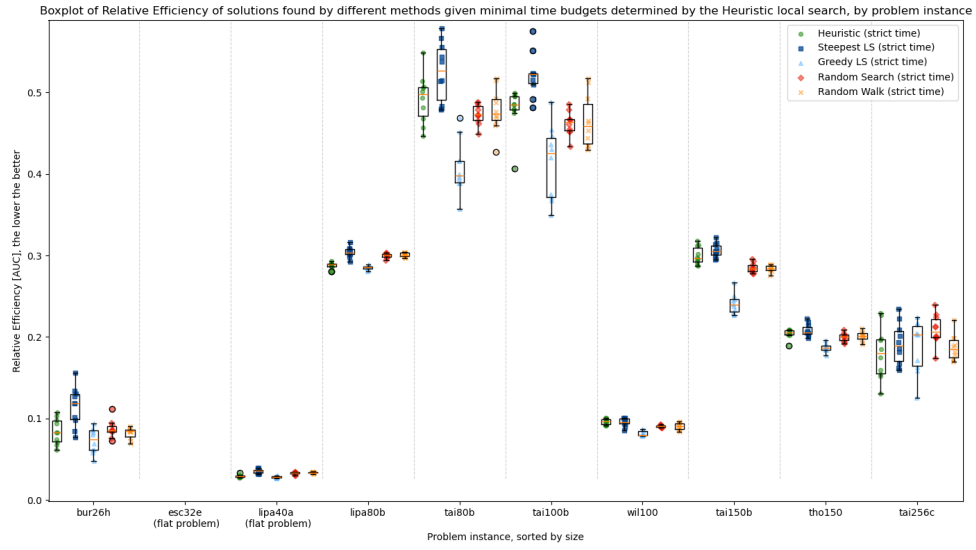


Figure 4: (truncated) Relative Efficiency for minimal time budgets

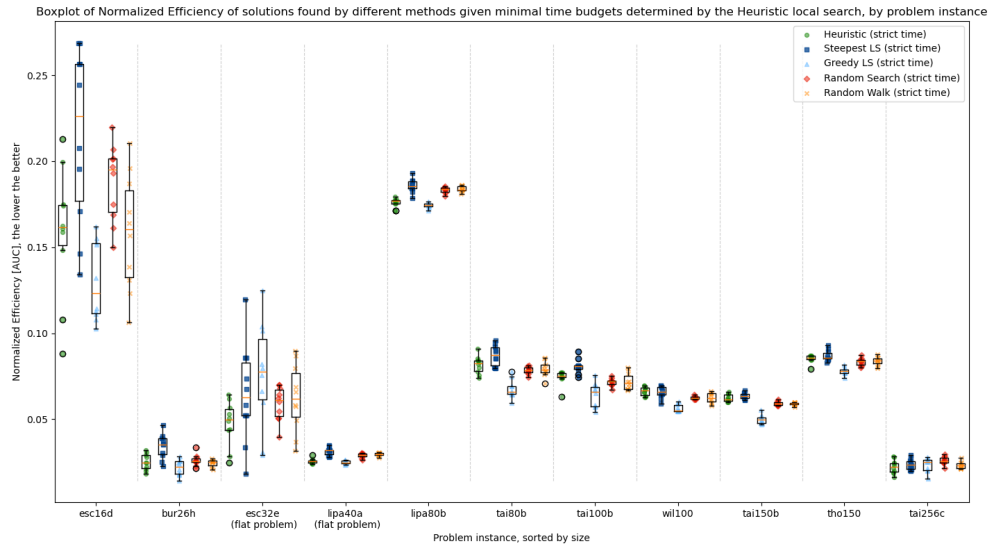


Figure 5: Normalized Efficiency for minimal time budgets

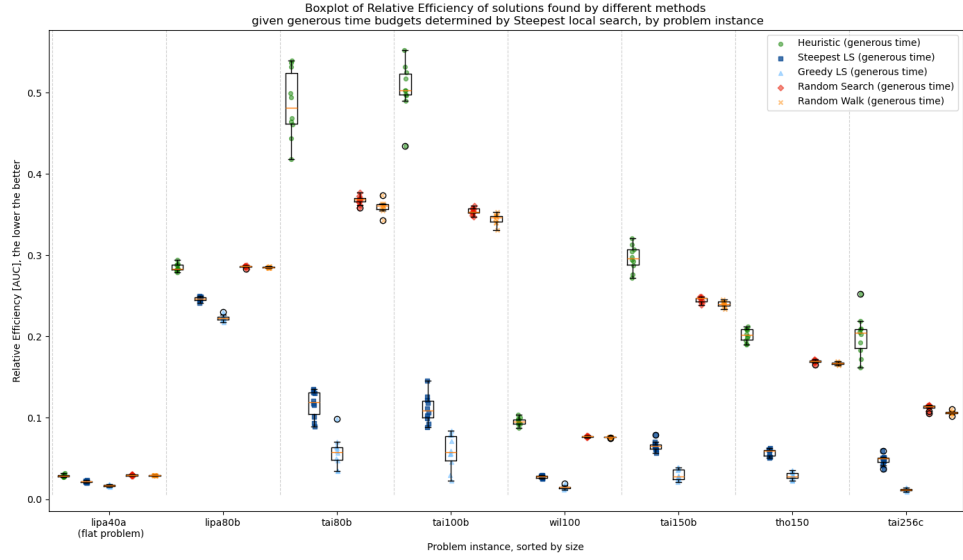


Figure 6: (truncated) Relative Efficiency for generous time budgets

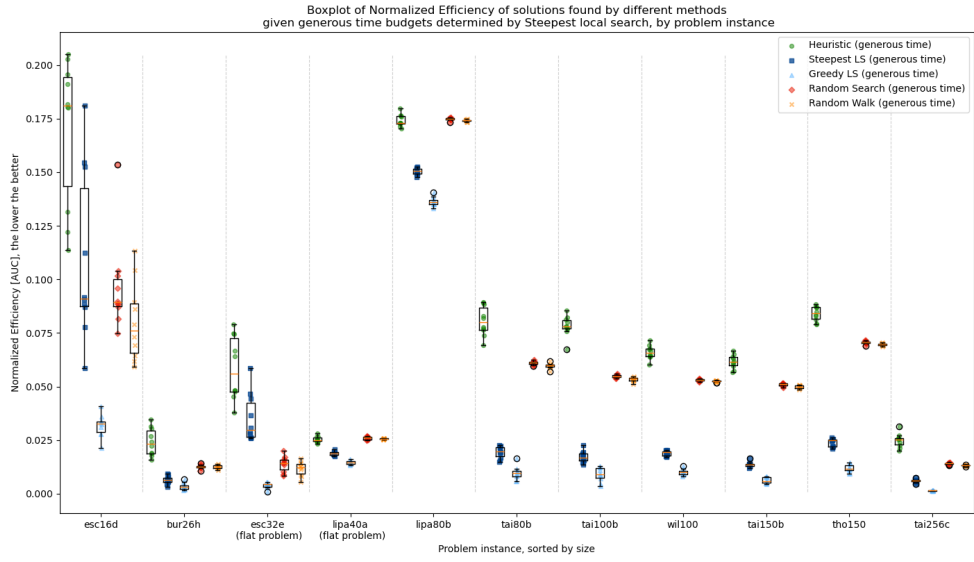


Figure 7: Normalized Efficiency for generous time budgets

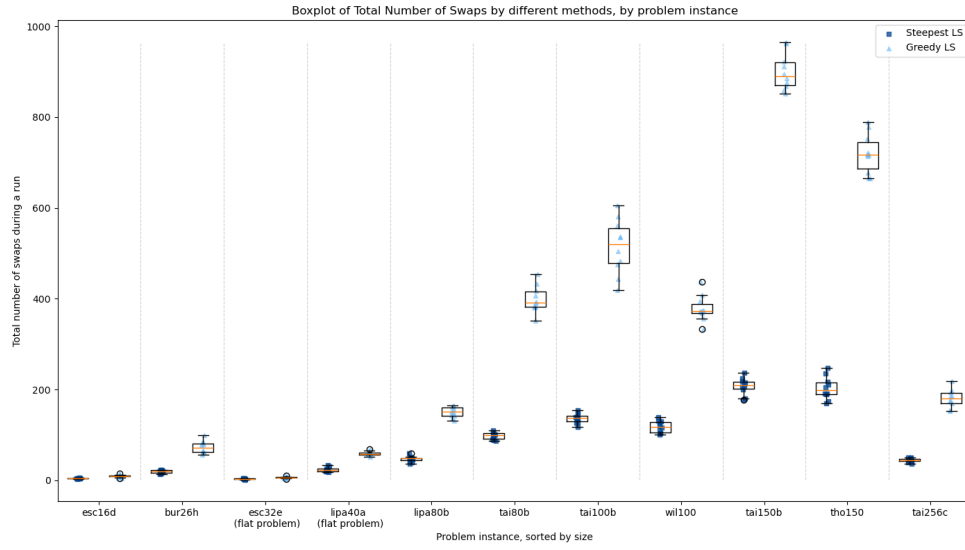


Figure 8: Total number of Swaps

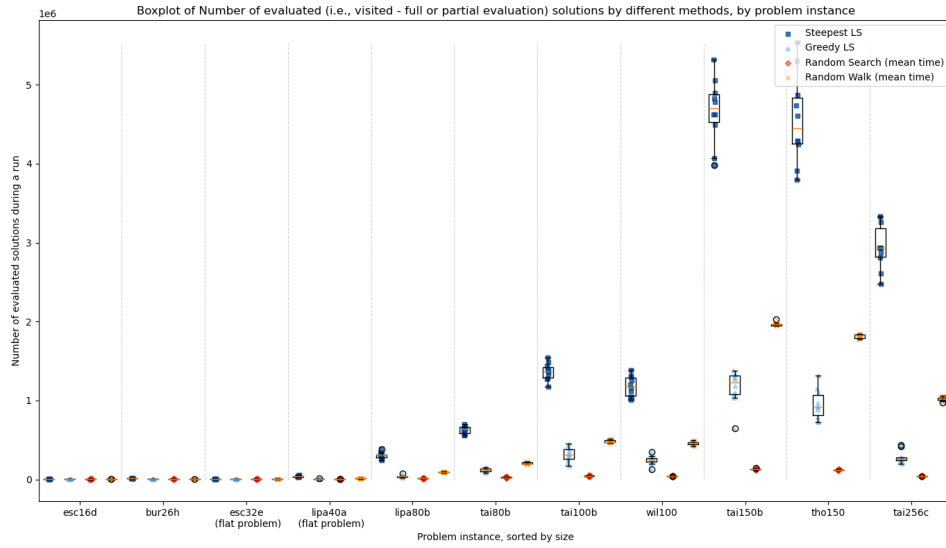


Figure 9: Number of evaluated solutions

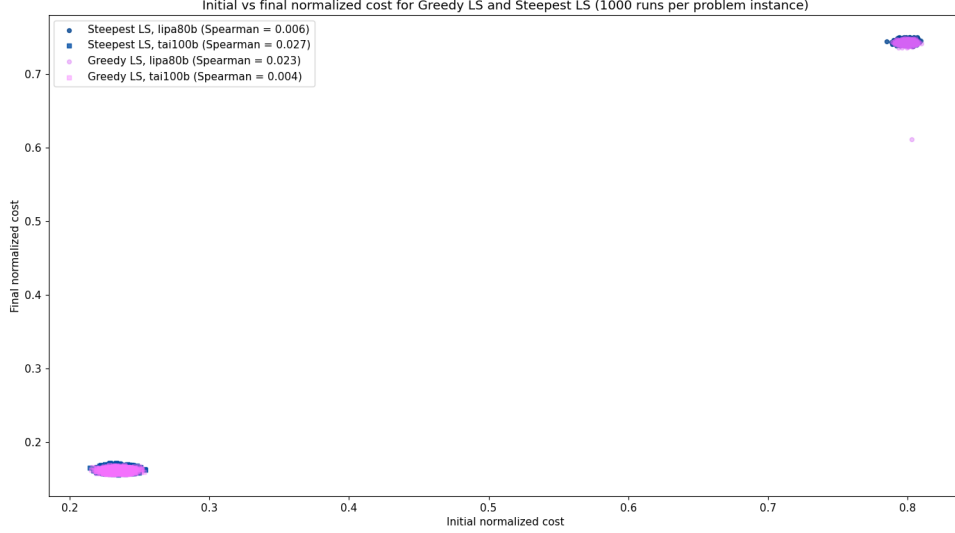


Figure 10: Initial vs Final Normalised cost of 1000 runs of **G** and **S**

3.6 Conclusions

Given all the evidence demonstrated so far, one can conclude the following:

1. Exploratory behaviour is beneficial when solving QAP.
2. Prior information should be utilized to make exploration more efficient.
3. Greedy strategy outperforms all the other algorithms (**H**, **S**, **RW**, **RS**).

4 Discovering the structure of the search space and the optimized function

The following instances were chosen for the analysis:

1. *lipa80b*, since all the methods demonstrated sub-optimal solutions compared to the optimal solution (which is equal to the identity permutation).
2. *tai100b*, due to noticeable variability in relative cost.

The other problems are not very interesting due to the lack of variability in final quality metrics. 1000 independent runs were performed of **G** and **S** for each of the problems above.

The following plots demonstrate relationships between initial and final quality. For this purpose *relative* and *normalised* cost can be used interchangeably when analyzing particular problems since each of the points on one plot can be linearly translated to the other plot. This doesn't hold for multiple problems. Spearman correlations are provided per each method and per each instance.

In both cases clouds of points were obtained without any visible structure. There is an isolated point (*outlier*) in *lipa80b* problem, which is equal to the optimal solution. This happened few times when thousands of runs were performed. The distinct feature of these clouds is that the cloud corresponding to **G** lies consistently lower, which supports the earlier conclusions that **G** performs better than **S**. Spearman correlations show close to 0 values (i.e. within -0.05.. +0.05 interval), meaning that there is no direct, monotonic "better in the beginning - better at the end" relationship between initial and final costs. Local search can map very different starts to very similar final costs. So runs with bad initial ranks and good initial ranks end up mixed in final ranks. Possible explanations of the observed low Spearman correlation are determined by landscape of the problems:

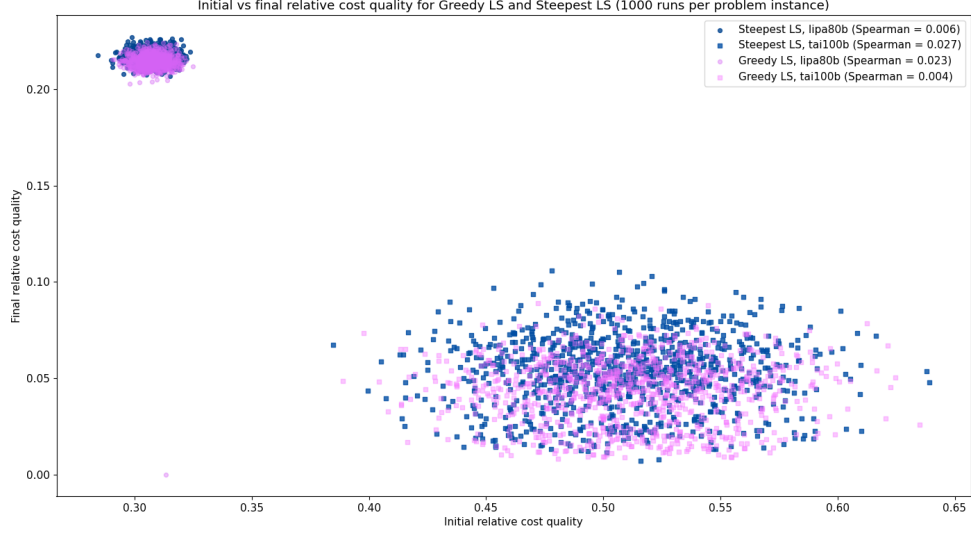


Figure 11: Initial vs Final Relative cost of 1000 runs of **G** and **S**

1. Multiple local minima/basins with different depths.
2. Large basins around some minima.
3. Plateaus or many near-ties in final quality.

5 Discovering the preferred number of repetitions of LS.

In this experiment, the horizontal axis is the number of independent starts $k=0..1000$, and for each k 2 values are plotted: *best-so-far relative cost quality* and *average-so-far relative cost quality* over the first k starts.

As was mentioned before, *lipa80b* has a significant improvement in the cost once during a few thousands restarts, and thus it makes sense to repeat **S** few thousands times (then the algorithm will find an optimal solution). At the same time *tai100b* demonstrates better improvement during few hundreds starts. Based on the visual characteristics of the **G** curve, it's better to stop early at $t = 200$ as plateau-based practical cutoff.

Hence, repeating LS is worthwhile, but the preferred number of restarts is instance- and algorithm-dependent: moderate budget is proposed for *tai100b*, and larger budget for *lipa80b* when chasing rare best-case gains.

6 Objective assessment of the similarity of locally optimal solutions found

Cayley similarity between two permutations π_1 and π_2 is defined through *Cayley distance* which is the minimum number of swaps necessary to obtain π_2 from π_1 :

$$\begin{aligned} \text{Cayley_sim}(\pi_1, \pi_2) &:= 1 - \text{Normalized Cayley distance}(\pi_1, \pi_2) = \\ &= 1 - \frac{1}{N-1} \min |\{k : s_k = \text{swap}_{i_k j_k}, \pi_2 = (\prod_k s_k) \cdot \pi_1\}| \end{aligned}$$

This similarity measure is proposed due to the natural definition of a metric on the space of permutations of length N (*permutohedron*) as it reflects how far are two permutations in the solutions space.

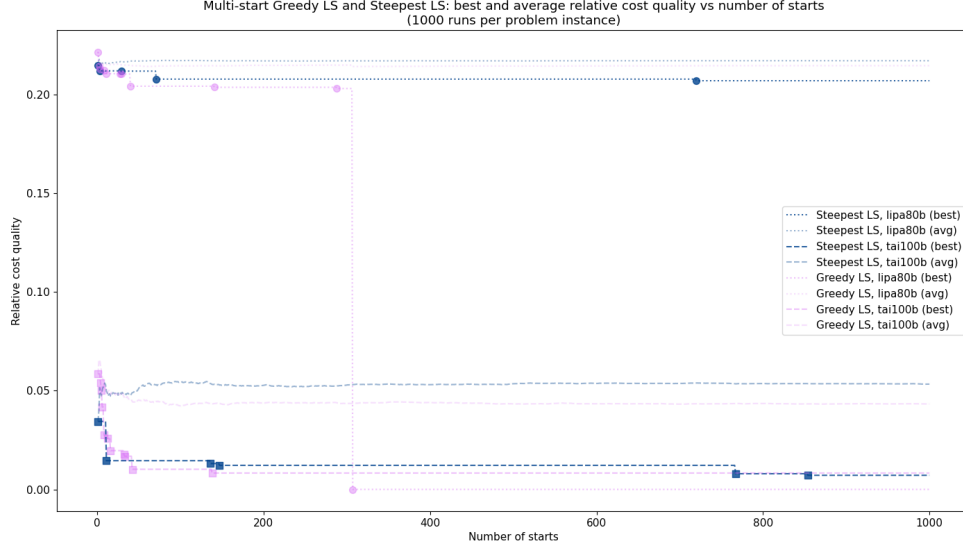


Figure 12: Best and Average relative cost plot for 1000 independent runs

Since both *Cayley* and *Hamming distances* are valid mathematical metrics, these similarity functions, for a given problem size N , naturally map distances into similarities.

The following two plots demonstrate relationship between Cayley similarity and *relative cost* (as it is more informative for multiple problems plots):

1. Data with a found outlier in *lipa80b*:
2. Data without an outlier, obtained by new 1000 restarts of algorithms:
- 3.

The figures demonstrate that there is from *moderate to strong linear relationship* between the similarity of a found solution (local optima) and the quality in *tai100b* problem. However, no such conclusion can be made in the case of *lipa80b* as this problem is likely to have only one distinct global optima (isolated point on the plot, which corresponds to the identity permutation). At the same time *tai100b* seems to have a lot of local optima with cost close to the best known one, and the closer these local optima to the best known solution, the lower the cost.

The following heatmaps show relationships between separate positions, matched with the positions of the best known solution, and the quality of the solution (same for both *relative cost* and *normalized cost* quality metrics).

Hence *lipa80b* has no particular position in the solution which would be responsible for a significant cost reduction, however there are few of such positions in *tai100b* (namely all of the positions from 1 to 31 have low negative correlation with the cost, and positions 27,29, and 30 have moderate negative correlation with the cost).

Note: The best-known solutions from *.sln* files use 1-based indices, which were converted to 0-based before computing similarity - when loading data into *Problem* class, and then passing the solution into *Permutation* structure.

7 Conclusions

7.1 General

- **QAP is NP-hard with a rugged landscape.** Pure random methods (**RS**, **RW**) are insufficient to find competitive solutions even for moderate-sized instances.

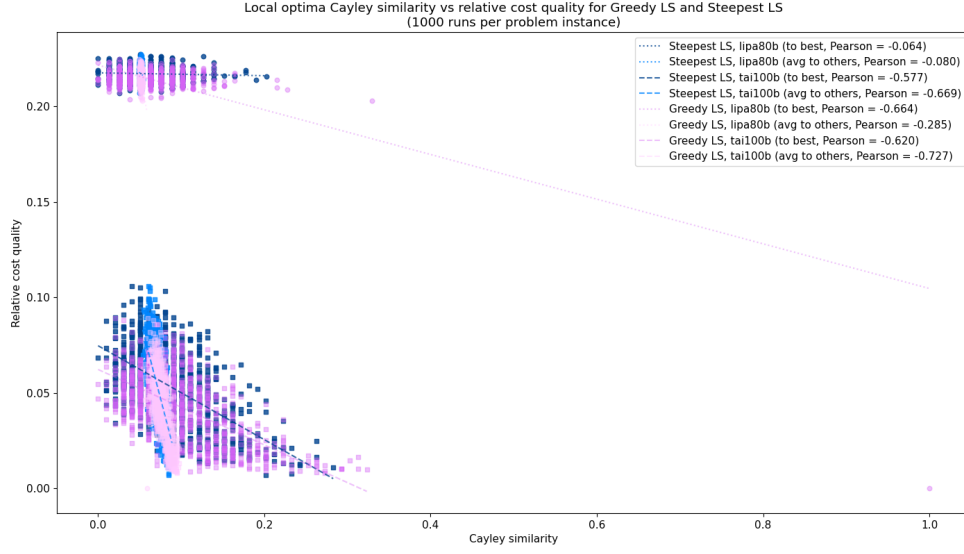


Figure 13: Cayley similarity vs. Relative Cost on data with an outlier in *lipa80b*

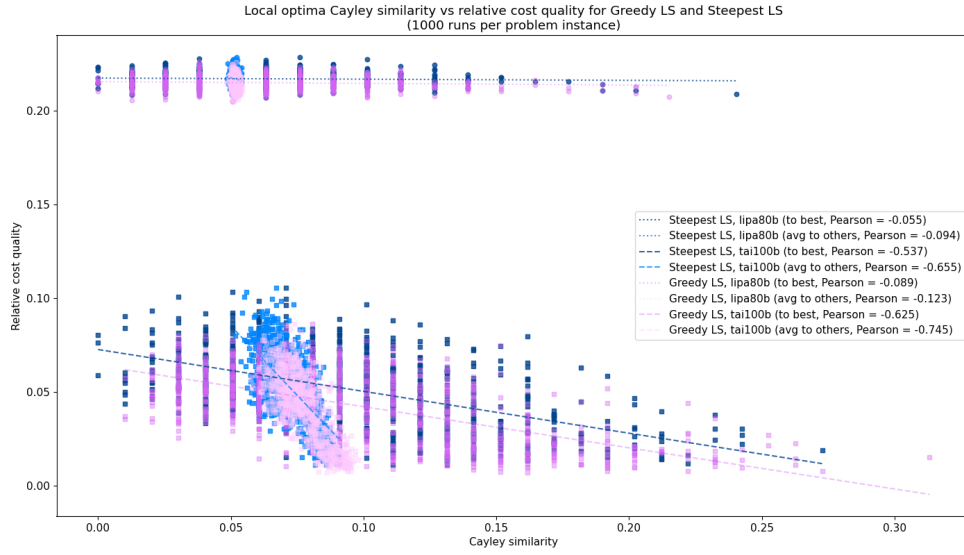


Figure 14: Cayley similarity vs. Relative Cost on data without outlier in *lipa80b*

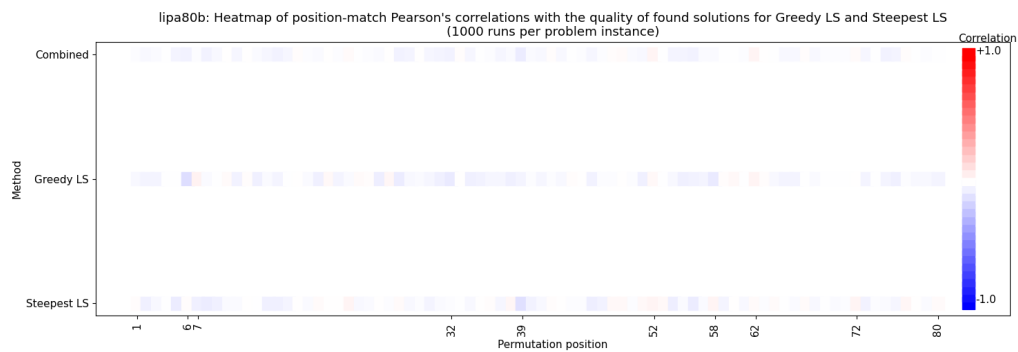


Figure 15: *lipa80b*: Dependency of position-matches with the best known solution and the quality of solutions

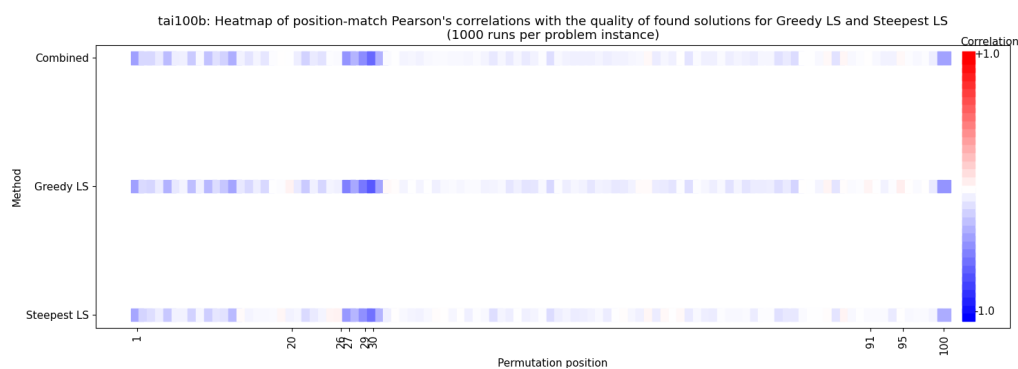


Figure 16: *tai100b*: Dependency of position-matches with the best known solution and the quality of solutions

- **Problem structure matters:** it strongly influences algorithm behaviour. "Flat" instances (*lipa40a*, *esc32e*) are degenerate cases where either a near-optimal solution is trivial to construct or cost improvement is structurally infeasible.

7.2 Specific

- **Algorithm ranking:** Local search methods (**G**, **S**) significantly outperform **H**, **RS**, **RW** on solution quality across all instances. **G** outperforms **S** in both quality and efficiency under all tested time budgets, attributed to its exploratory behaviour enabling broader landscape coverage per unit time.
- **Initial solution irrelevance:** Near-zero Spearman correlation between initial and final cost (within ± 0.05) across 1000 runs confirms that initial solution quality does not predict final quality, which is consistent with a landscape of many basins of varying depth.
- **Multi-start strategy:** Repeating LS is worthwhile but the optimal restart budget is instance-dependent:
 - around 200 restarts suffice for **G** on *tai100b*;
 - several thousand are needed for **S** on *lipa80b* to encounter the rare optimal basin.
- **Search space structure (similarity analysis):**
 - *tai100b* exhibits a structured landscape where closer local optima (higher Cayley similarity to the best known) have lower cost (Pearson correlation between $\mathbf{j}$ -0.6)
 - *lipa80b* has a near-flat landscape with a single isolated global optimum (identity permutation), with no such correlation. Specific positions 27–30 in *tai100b* account for disproportionate cost reductions.

8 Challenges encountered

1. Debugging *Matplotlib-cpp* errors in C++ was difficult and time-consuming.
2. Containerization and dependency setup (C++/Python/plotting) required significant effort.
3. Refactoring core structures introduced temporary performance regressions.
4. *int* overflow caused invalid results (e.g., negative costs), so wider integer types were needed.
5. Parallelization improved speed but complicated fair benchmarking and determinism.
6. Final figures (with low number of restarts) required strict reproducibility settings (single-core runs, fixed seeds).
7. Randomness quality of *reshuffle* and *get_random_pair* had to be validated; two Fisher-Yates passes worked better in practice.

9 Justification and Future Improvements

Justification. The introduced improvements were necessary to make results valid, comparable, and efficient:

1. Fixed arithmetic issues (overflow) to avoid invalid objective values.
2. Improved reproducibility (fixed seeds, controlled runs) for fair method comparison.
3. Improved random operators quality, which stabilized multi-start behavior.
4. Optimized core implementation to run larger experiments in the same time.

Future improvements and expected effects:

1. Better initialization heuristic (instead of matching single elements rows or columns with the highest/lowest norms can be matched) *for* better starting solutions.
2. Additional caching in cost/delta-cost loops *for* faster iterations.
3. Adaptive restart budget instead of fixed restarts *for* better time-quality trade-off.
4. Hybrid methods (e.g., LS + diversification/perturbation) *for* better escape from local minima.
5. Parallel independent restarts *for* lower wall-clock time.

References

- [1] Maciej Komosiński and Maciej Hapke. “Optimization – local search. Lecture notes”. In: (2023). URL: <https://www.cs.put.poznan.pl/mkomosinski/lectures/optimization/LS-en.pdf>.
- [2] “Quadratic assignment problem”. In: *Wikipedia* (). URL: https://en.wikipedia.org/wiki/Quadratic_assignment_problem.
- [3] S.E. KARISCH R.E. BURKARD E. ÇELA and F. RENDL. “QAPLIB – A Quadratic Assignment Problem Library”. In: ().
- [4] *Matplotlib C++*. URL: <https://github.com/lava/matplotlib-cpp>.
- [5] Thomas Stutzle Irina Dumitrescu. “Topics in Local Search”. In: (). URL: <https://www.cs.put.poznan.pl/mkomosinski/lectures/optimization/extras/QAP-fast-update.pdf>.